TECHNICAL GUIDE · DEMO MANUAL

BIZRO

THE DEEP GUIDE

How a WhatsApp voice note becomes a lender-legible credit record: local setup, the webhook pipeline, the Qwen model stack and its fallbacks, the demo walkthrough, what is real versus mock, the security posture, and operations. Every claim verified against the code or the live deployment.

QWEN3-ASR-FLASH QWEN-FLASH QWEN-VL-OCR

CONTENTS

This guide was generated from the verified working manual on 4 September 2026. Every file path, command, model name and status code in it was checked against the repository or the live deployment on that date. The editable source lives at `docs/GUIDE.md` and the repo overview at `README.md`.

1 · LOCAL SETUP, END TO END

Prerequisites

- Python 3.12+ (the repo was developed on 3.14), Node 18+, Git Bash on Windows or any POSIX shell.
- Optional: a DashScope API key (Alibaba Cloud Model Studio) for live model calls, WhatsApp Cloud API credentials for live messaging, a Neon Postgres URL. **Zero credentials is a valid state** — everything runs mocked and clearly labeled.

Install and configure

```
python -m venv .venv
source .venv/Scripts/activate    # Git Bash on Windows
python -m pip install -r requirements.txt
cp .env.example .env
```

The root `requirements.txt` is the union of the server and the three agent packages (FastAPI, SQLAlchemy, httpx, pydantic, edge-tts, ffmpeg via imageio-ffmpeg, pytest). Model credentials are env-only — see the table in `README.md` and `.env.example`.

Run the server

```
python credit-agent/scripts/seed_demo.py    # demo merchants, only if DB empty
python -m uvicorn server.app.main:app --reload --port 8000
```

The server binds paths to the repo root, so it can be started from anywhere. On boot it logs which integrations are live vs mock (and never logs the database password — see `_safe_db_label` in `server/app/main.py`). Check http://localhost:8000/health for the same information in JSON.

With the frontends built (`npm run build` in `dashboard/` and `site/`), the server alone serves the landing page at `/` and the dashboard at `/ledger` — exactly the production topology (`server/app/main.py`, static router at the bottom of the file). If no dashboard build exists, the SPA fallback returns a plain instruction message instead of a 500.

Run the dashboard and site dev servers

```
cd dashboard && npm install && npm run dev   # http://localhost:5173
cd site && npm install && npm run dev        # second terminal
```

Both Vite configs proxy API routes to the backend on `:8000`: `dashboard/vite.config.ts` forwards `/api` and `/webhook`; `site/vite.config.ts` additionally forwards `/ledger`, `/credit`, `/settings` and `/assets` so cross-app links work in dev. Keep the server running — the dev proxies expect it.

One-command alternative

`bash scripts/run_demo.sh` does all of the above: venv, install, UI builds (skip with `BIZRO_SKIP_UI=1`), seeding when the DB is empty, then serves on `:8000`.

2 · THE WHATSAPP WEBHOOK FLOW

Entry point: `server/app/webhook.py`. `GET /webhook/whatsapp` is the Meta verification handshake (constant-time token compare). `POST /webhook/whatsapp` is the ingest path. Step by step for a voice note:

1. **Signature check.** With `WHATSAPP_APP_SECRET` set (it is, in production), every request must carry a valid `X-Hub-Signature-256` (HMAC-SHA256 of the raw body). Verification lives in `server/app/whatsapp_client.py` (constant-time compare). Unsigned posts get 403 — with one carve-out: the simulator envelope is accepted unsigned, and only for the three public demo numbers `923001234567`, `923009999888`, `923009111222`.
2. **Dedup.** Meta redelivers. The `wamid` is claimed in `processed_messages` before any processing; a repeat delivery is acknowledged as deduped and never re-processed.
3. **Merchant upsert** by `wa_id`. A WhatsApp profile name is only used at account creation — it is attacker-controlled text and can never rename an existing merchant's ledger.
4. **Media bytes.** Either from the simulator envelope (`bizro_sim.media_b64`, base64 inline — this is what the landing-page mic and the dashboard simulator send), or the real two-step Graph API download (metadata, then bytes) in `server/app/whatsapp_client.py`.
5. **Validation.** Size caps (16 MB audio, 5 MB image) and magic-byte sniffing in `server/app/media.py`. An Ogg page labeled as a photo, or an executable, is rejected with a polite reply; nothing is stored.
6. **Storage — the audit trail.** Bytes land at `media/<yyyy>/<mm>/<uuid>.<ext>` plus a `media_blobs` row with the sha256 and a durable BYTEA copy of the bytes (serverless disk is ephemeral; the database copy survives cold starts). `GET /api/media/{id}` serves from disk when present, else straight from Postgres, else 410 (`server/app/api.py`).
7. **Pipeline.** `server/app/dispatch.py` lazily imports `voice_agent.pipeline.process_voice_note` (audio) or `vision_agent.pipeline.process_receipt_image` (photo). Receipts also get the merchant's recent expense history so the vision pipeline can raise price-anomaly and duplicate-suspect flags. If an agent package were missing, a clearly-labeled synthetic server fallback runs instead (mock marker, sub-threshold confidence, entry stays pending) — never silently.
8. **Persist.** Pipeline output is validated as a schema transaction and written with full provenance: `source_type`, `source_model`, `confidence`, `raw_model_output` are immutable; the first human edit snapshots the original values into `transactions.original_values`. Confidence below `CONFIDENCE_CONFIRM_THRESHOLD` (0.75) forces `status=pending` until the merchant confirms.
9. **Reply.** The confirmation text is stored as an `outbound_messages` row and delivered via the Graph API — as an interactive message with one-tap Correct/Edit buttons while the entry is pending. Delivery failure never discards the saved entry (demo numbers are outside Meta's test

allow-list, so this path is exercised constantly).

Text messages follow a parallel branch: confirm/reject words (`1`/`0`, English or Urdu) act on the latest pending entry; onboarding triggers get the welcome sequence; anything else is parsed as a free-text transaction exactly like a voice transcript. Button presses (correct/edit payloads) act on the latest pending entry too. Every reply — confirmation, clarification, rejection, onboarding — gets an `outbound_messages` row; nothing reaches the merchant without one.

Failure classes stay honest. A parse miss (no amount, non-transaction, schema violation) sends a clarification and persists nothing; a genuine model timeout or outage sends a busy reply and persists nothing. The two never swap copy.

3 · THE QWEN MODEL STACK

Everything runs on Alibaba Cloud Model Studio via the DashScope OpenAI-compatible endpoint (`/chat/completions`). Production uses the international base URL `https://dashscope-intl.aliyuncs.com/compatible-mode/v1` (visible in `GET /health`). The thin client is `server/app/dashscope_client.py` for the server and per-agent copies for the pipelines.

- **Speech to text — qwen3-asr-flash.** `STT_PROVIDER=qwen` sends the audio as a base64 data-URI `input_audio` content item in a chat-completions call (`voice-agent/voice_agent/stt_client.py`). This is the production primary.
- **STT fallback — Groq Whisper.** Any qwen3-asr-flash failure (container format rejection, quota, network — anything) falls back automatically to `whisper-large-v3-turbo` on Groq (free tier, `STT_API_KEY`). A voice note is never lost to a transcriber hiccup; the fallback is logged.
- **Transaction parsing — qwen-flash** (`MODEL_VOICE`). One prompt produces the structured transaction JSON (Urdu-aware: phonetic English number words, mixed script). One repair-retry fires if the output fails schema validation (`voice-agent/voice_agent/pipeline.py`). Typed WhatsApp text messages go through the same parser with `source.type="text"`.
- **Receipt OCR — qwen-vl-ocr** (`MODEL_OCR_VL`, selected by `OCR_MODEL=vl`). `vision-agent/vision_agent/adapters.py` keeps a second real adapter, qwen3.5-ocr (`MODEL_OCR_NEW`), for the bake-off; the winner is an env switch, reversible and auditable. Unparseable model output is surfaced as an unreadable extraction and a polite retry — the pipeline never guesses.
- **Credit narrative — qwen-flash** (`MODEL_REASONING`). The score itself is a deterministic rubric over real aggregates (`credit-agent/credit_agent/rubric.py`); the model writes the short narrative stored in `credit_reports`. The stored key is `narrative_ur` — a historical name; the text is simple English by design (`credit-agent/credit_agent/narrative.py`).

Mock semantics (`MOCK_MODE` in `server/app/config.py`): `auto` = real calls when the key exists, clearly-labeled synthetic output otherwise; `always` = always mock (demo-safe); `never` = refuse to fake (a missing key raises). Every mock payload carries `"mock": true`. `llm_guard.py` at the repo root counts live calls and can hard-stop at a daily budget (used when the endpoint points at OpenRouter's free tier instead of DashScope).

4 · DEMO WALKTHROUGH

The seeded merchants

STORE	WA_ID (PUBLIC DEMO NUMBER)	ROLE IN THE DEMO
Al-Madina Kiryana Store	`923009999888`	The healthy case: 88 entries June–September 2026 (count verified against the live API on 2026-09-04), regular sales/udhar/receipt mix, a real readiness verdict.
Bilal Ki Dukan	`923009111222`	The contrast case: sparse logging with a gap, to show what "not ready" looks like.

Seeded by credit-agent/scripts/seed_demo.py; both numbers are public simulator constants.

`GET /api/merchants` orders the healthy store first, so a judge landing on `/ledger` or `/credit` opens on the full story, not the thin sandbox. Any real merchant's number is reduced to a last-4 hint on that endpoint.

The simulator surfaces

- **Dashboard** `/simulator` — a WhatsApp-shaped chat backed by real API calls: send a note, watch the parse land in the ledger, reply `1` to confirm, poll `GET /api/merchants/{id}/outbound` for Bizro's replies. Confirmation bubbles lazily get a stamped invoice image (rendered, stored as a normal media blob, pinned into the outbound row — `_ensure_invoice_media` in `server/app/api.py`).
- `server/scripts/simulate_inbound.py` — posts simulator envelopes from the CLI (default sandbox number `923001234567`).
- `server/scripts/demo_flow.py` — the rehearsal driver: seeds if empty, drives the whole story over real HTTP (or in-process with `--local`), prints a judge-facing timeline with per-step timings, and exits non-zero the moment any step fails. Rehearse with it, not on stage.
- **Landing-page hero** (`site/src/hero-demo.ts`) — the "Tap to record a note" card records real browser audio (MediaRecorder) and POSTs it to the live `/webhook/whatsapp` in the exact simulator envelope. The invoice card then renders what the server actually parsed (kind, amount, counterparty, confidence); missing fields render as unknown, mock markers as-is.

Suggested two-minute flow

1. Land on https://bizro-pk.vercel.app — hero card: record a short note ("sold ghee to Rahmat for three thousand cash"), stop, watch the parsed invoice appear.
2. Open `/ledger` — Al-Madina's four months of history; tap a demo-marked row, open its audit trail (the original media, model, confidence).
3. Open `/credit` — the Credit Readiness report for Al-Madina, then switch to Bilal Ki Dukan for the contrast.

4. Open `/simulator` — send a fresh note live, confirm with `1`.

5 · WHAT IS REAL VS MOCK

The honesty law. Every mock is labeled, every failure is visible, and nothing synthetic is ever presented as a real model result. This rule governs the whole codebase (referenced in code as STATUS.md D0-3).

Real in production right now:

- The WhatsApp Cloud API webhook and outbound sends (System User token, signature enforced — `/health` reports both live).
- Neon Postgres persistence: merchants, transactions with provenance, outbound log, credit reports, and the media bytes themselves.
- All model calls on DashScope: qwen3-asr-flash (STT), qwen-flash (parse, narrative), qwen-vl-ocr (OCR). `/health` shows the live base URL.
- The audit trail: original audio/photo bytes served back from `/api/media`.
- Urdu voice replies via edge-tts (`POST /api/tts`, voice `ur-PK-AsadNeural`).

Mock or seeded, and labeled as such:

- The two seeded demo merchants and the sandbox ledger — dashboard rows carry a visible "Demo data — seeded example entries" marker (`dashboard/src/components/MockBanner.tsx`).
- Server-fallback pipeline output when an agent package is absent (`server/app/dispatch.py`) — mock-flagged, low confidence, stays pending.
- Any model call made without a key under `MOCK_MODE=auto` — every payload carries `"mock": true`, and the landing-page hero displays the marker verbatim instead of hiding it.
- WhatsApp sends to demo numbers are refused by Meta (outside the test allow-list); the entry stays saved and the UI shows "saved, not delivered" rather than pretending the message arrived.

6 · SECURITY POSTURE

SETTING	DEFAULT	WHY
`RATE_LIMIT_WEBHOOK_PER_HOUR`	30 / hour	The webhook triggers paid AI calls — a tight per-IP hourly budget.
`RATE_LIMIT_GENERAL_PER_MIN`	120 / minute	Everything else, per IP.
`CONFIDENCE_CONFIRM_THRESHOLD`	0.75	Parses below this stay pending until the merchant confirms.
Media caps (audio / image)	16 MB / 5 MB	Enforced with magic-byte sniffing before anything touches disk.

All limits are env-tunable per request; 429s carry Retry-After.

- **Webhook signature.** `X-Hub-Signature-256` HMAC-SHA256 over the raw body, constant-time compare, enforced whenever `WHATSAPP_APP_SECRET` is set. The only unsigned path is the simulator envelope, and only for the three public demo numbers — a forged `bizro_sim` marker from any other `wa_id` is still a 403.
- **Headers and CSP.** Every response carries `nosniff`, `X-Frame-Options: DENY`, a strict referrer policy, a permissions-policy limited to the app's own camera/mic use, and a CSP of `default-src 'self'` with no third-party script origins (`server/app/middleware_security.py`).
- **Inbound media.** Size caps and magic-byte sniffing before anything touches disk; executables and mismatched content types are rejected.
- **Untrusted model output.** Parsed strings are length-capped before they enter prompts; transaction JSON is marked UNTRUSTED in the reminder-draft prompt; HTML-escaping on rendered artifacts; the CSV export neutralizes formula injection (`=`/`+`/`-`/`@` prefixes) and is UTF-8-BOM + CRLF so Excel opens Urdu correctly (`server/app/api.py`).
- **Privacy.** Real merchant phone numbers never appear on public endpoints (last-4 hint only); the database DSN is logged without its password.

7 · OPS

- **Deploy.** `bash scripts/deploy.sh` builds both frontends, runs `vercel deploy --prod`, re-points the four aliases (`bizro-pk`, `getbizro`, `bizro-app`, `bizro-ai .vercel.app`), and curls `/health` and `/` for 200s. Prereq: `npx vercel login` once, env vars set in the Vercel dashboard.
- **Vercel shape.** `api/index.py` wraps the FastAPI app as one serverless function (region `sin1`, 60 s max duration — `vercel.json`); static assets route to `site/dist` and `dashboard/dist`. On serverless, `MEDIA_DIR` is `/tmp/media` and SQLite would be ephemeral — hence Neon.
- **Durable media.** Every media blob is stored twice: the dated filesystem tree and a BYTEA column in Postgres. `/api/media/{id}` prefers disk, falls back to the database across cold starts, and 410s only if both are gone.
- **Observability.** `GET /health` reports integration live/mock status, signature enforcement, pipeline import status, and the confidence threshold — the same check the deploy script and the QA live walk (`qa/live_walk/live_walk.py`) use.

8 · TROUBLESHOOTING

- **403 "invalid signature" on your own POSTs** — you set `WHATSAPP_APP_SECRET`, so unsigned posts are rejected outside the demo sandbox. Use the simulator envelope with a demo `wa_id`, or sign the body.
- **Everything says "mock"** — no `DASHSCOPE_API_KEY` in the environment, or `MOCK_MODE=always`. `/health` tells you which integration is mock and why.
- **The "dashboard not built" message at `/ledger`** — run `npm install && npm run build` in `dashboard/`, or use its dev server.
- **Log line "Qwen ASR failed … falling back"** — expected resilience, not an error: the note was transcribed by Whisper instead. Frequent fallbacks usually mean a container format qwen3-asr rejects; the decode step (`voice-agent/voice_agent/decode.py`) normalizes most formats first.
- **429s during local testing** — the per-IP limiter; raise `RATE_LIMIT_GENERAL_PER_MIN`/`RATE_LIMIT_WEBHOOK_PER_HOUR` in the env, or restart the server (windows reset on boot).
- **Reports 500 only in production** — historically the Neon password being masked when passed to credit-agent; the fix lives in `server/app/dispatch.py` (`render_as_string(hide_password=False)`). If it reappears, start there.
- **TTS returns 502** — honest failure (budget/network); the frontend skips audio rather than faking it. Retry, or check `MEDIA_DIR` writability.
- **A Meta redelivery created a duplicate entry** — it should not: dedup is keyed on `wamid` in `processed_messages`. If you see duplicates, check that table and the claim logic at the top of the POST handler in `server/app/webhook.py`.